

HOJA DE TRABAJO

Práctica — Examen Final

Tipo A — Traducción a LLVM IR · Tipo B — Interpretación de IR · Tipo C — Optimización de IR

TIPO A — Traducción a LLVM IR

Traduce cada función al LLVM IR equivalente usando el patrón `alloca + load + store`. Incluye etiquetas en cada bloque básico y el terminador (`br` o `ret`) al final de cada uno.

A.1 — `diferencia_abs(a, b)`

Traduce la siguiente función a LLVM IR completo con bloques básicos etiquetados.

```
def diferencia_abs(a: int, b: int)
    if a >= b:
        return a - b
    else:
        return b - a
```

A.2 — `clasificar(n)`

Traduce la función de clasificación de signo. Presta atención a los bloques encadenados.

```
def clasificar(n: int)
    if n > 0:
        return 1
    elif n < 0:
        return -1
    else:
        return 0
```

A.3 — `saturar(n)` — mayor complejidad

Traduce la función. Combina dos comparaciones encadenadas que filtran un rango.

```
def saturar(n: int) -> int:
    if n < 0:
        return 0
    elif n > 100:
        return 100
    else:
        return n
```

TIPO B — Interpretación de LLVM IR

Para cada fragmento de LLVM IR: (a) describe en una oración qué computa la función y (b) escribe el pseudocódigo equivalente.

B.1 — producto(a, b)

Analiza el IR e interpreta qué operación aritmética realiza la función.

```
define i32 @producto(i32 %a, i32 %b) {
entry:
    %ptr_r = alloca i32
    %ptr_i = alloca i32
    store i32 0, i32* %ptr_r
    store i32 0, i32* %ptr_i
    br label %cond
cond:
    %i = load i32, i32* %ptr_i
    %c = icmp slt i32 %i, %b
    br i1 %c, label %body, label %exit
body:
    %r = load i32, i32* %ptr_r
    %r2 = add i32 %r, %a
    store i32 %r2, i32* %ptr_r
    %i2 = add i32 %i, 1
    store i32 %i2, i32* %ptr_i
    br label %cond
exit:
    %res = load i32, i32* %ptr_r
    ret i32 %res
}
```

B.2 — secreto(n)

```
define i32 @secreto(i32 %n) {
entry:
    %ptr_s = alloca i32
    %ptr_i = alloca i32
    store i32 0, i32* %ptr_s
    store i32 0, i32* %ptr_i
    br label %cond
cond:
    %i = load i32, i32* %ptr_i
    %c = icmp sle i32 %i, %n
    br i1 %c, label %body, label %exit
body:
    %s = load i32, i32* %ptr_s
    %s2 = add i32 %s, %i
    store i32 %s2, i32* %ptr_s
    %i2 = add i32 %i, 2
    store i32 %i2, i32* %ptr_i
    br label %cond
exit:
    %res = load i32, i32* %ptr_s
    ret i32 %res
}
```

B.3 — proceso(n)

```
define i32 @proceso(i32 %n) {
entry:
  %ptr_r = alloca i32
  %ptr_i = alloca i32
  store i32 0, i32* %ptr_r
  store i32 0, i32* %ptr_i
  br label %cond
cond:
  %i = load i32, i32* %ptr_i
  %c = icmp slt i32 %i, %n
  br i1 %c, label %body, label %exit
body:
  %r = load i32, i32* %ptr_r
  %r2 = add i32 %r, %n
  store i32 %r2, i32* %ptr_r
  %i2 = add i32 %i, 1
  store i32 %i2, i32* %ptr_i
  br label %cond
exit:
  %res = load i32, i32* %ptr_r
  ret i32 %res
}
```

TIPO C — Optimización de LLVM IR

Para cada ejercicio: (1) identifica las técnicas aplicables y la línea que las activa, y (2) escribe el IR optimizado resultante. Aplica al menos 2 técnicas por ejercicio.

Referencia rápida — técnicas de optimización:

Técnica	Patrón que la activa	Transformación
Plegado de constantes	add i32 6, 4 (ambos literales)	→ reemplazar por el literal 10
Simplif. algebraica	x*1, x+0, x-x, 0+x, 1*x	→ x, x, 0, x, x respectivamente
Reducción de potencia	mul x, 2^k (k = 1, 2, 3 ...)	→ shl x, k (desplazamiento izq.)
CSE (subexp. común)	mul %a,%b ... mul %a,%b (dos veces)	→ reusar el primer resultado
Promoción de memoria	store %v, %ptr luego load %ptr (inmediato)	→ load superfluo, %va = %v directamente
Elim. código muerto	resultado = 0 o nunca usado después	→ eliminar la instrucción completa

C.1 — transformar(n)

```
define i32 @transformar(i32 %n) {
entry:
  %a = add i32 6, 4           ; (1)
  %b = mul i32 %n, 1         ; (2)
  %c = add i32 %b, 0         ; (3)
  %d = sub i32 %c, %c         ; (4)
  %e = mul i32 %a, %c         ; (5)
  %f = add i32 %e, %d         ; (6)
```

```
    ret i32 %f
}
```

C.2 — rapido(x)

```
define i32 @rapido(i32 %x) {
entry:
    %a = mul i32 %x, 4          ; (1)
    %b = mul i32 %x, 8          ; (2)
    %c = add i32 0, %a          ; (3)
    %d = mul i32 1, %b          ; (4)
    %e = sub i32 %d, %b         ; (5)
    %f = add i32 %c, %e         ; (6)
    ret i32 %f
}
```

C.3 — simplificar(a, b)

```
define i32 @simplificar(i32 %a, i32 %b) {
entry:
    %ptr_a = alloca i32        ; (1)
    %ptr_b = alloca i32        ; (2)
    store i32 %a, i32* %ptr_a  ; (3) escribe %a
    store i32 %b, i32* %ptr_b  ; (4) escribe %b
    %va = load i32, i32* %ptr_a ; (5) lee lo que se acaba de escribir
    %vb = load i32, i32* %ptr_b ; (6) lee lo que se acaba de escribir
    %r = add i32 %va, %vb       ; (7)
    %s = mul i32 %r, 4          ; (8)
    %t = add i32 %s, 0          ; (9)
    ret i32 %t
}
```

REFERENCIA RÁPIDA — Instrucciones LLVM IR

Instrucciones de uso frecuente:

Instrucción	Sintaxis ejemplo	Descripción
<code>alloca</code>	<code>%p = alloca i32</code>	Reserva espacio en stack
<code>store</code>	<code>store i32 %v, i32* %p</code>	Escribe valor en memoria
<code>load</code>	<code>%v = load i32, i32* %p</code>	Lee valor de memoria
<code>add / sub / mul</code>	<code>%r = add i32 %a, %b</code>	Aritmética entera
<code>shl</code>	<code>%r = shl i32 %x, 2 ; x * 4</code>	Desplaz. izq. ($\times 2^k$)
<code>icmp</code>	<code>%c = icmp slt i32 %a, %b</code>	Comparación (retorna i1)
<code>br condicional</code>	<code>br i1 %c, label %true, label %false</code>	Salto condicional
<code>br / ret</code>	<code>br label %bloque / ret i32 %v</code>	Salto incondicional / retorno

Predicados icmp con signo:

<code>slt</code> (<)	<code>sle</code> ($\leq$)	<code>sgt</code> (>)	<code>sge</code> ($\geq$)	<code>eq</code> (==)	<code>ne</code> ($\neq$)
----------------------	-----------------------------	----------------------	-----------------------------	----------------------	----------------------------

Patrón canónico — bucle while con alloca:

```
define i32 @funcion(i32 %n) {
preheader:
    %ptr_var = alloca i32                ; declarar variable
    store i32 <valor_inicial>, i32* %ptr_var ; inicializar
    br label %cond                       ; entrar al bucle
cond:
    %var = load i32, i32* %ptr_var        ; leer variable
    %c = icmp <pred> i32 %var, <limite>    ; evaluar condición
    br i1 %c, label %body, label %exit    ; bifurcar
body:
    ; ... cuerpo del bucle ...
    %new = add i32 %var, <paso>            ; actualizar variable
    store i32 %new, i32* %ptr_var        ; guardar nuevo valor
    br label %cond                       ; volver a condición
exit:
    %res = load i32, i32* %ptr_resultado
    ret i32 %res
}
```